



CMSC 414

Computer and Network Security

Lecture 14

Jonathan Katz



Mandatory access control



“Military security policy”

- ◆ Primarily concerned with secrecy
- ◆ Objects given “classification” (rank; compartments)
- ◆ Subjects given “clearance” (rank; compartments)
- ◆ “Need to know” basis
 - Subject with clearance (r, C) dominates object with classification (r', C') only if $r' \leq r$ and $C' \subseteq C$
 - Defines a partial order ... classifications/clearance not necessarily hierarchical



Security models

◆ Multilevel security

– Bell-LaPadula model

- Identifies allowable communication flows
- Concerned primarily with ensuring secrecy

– Biba model

- Concerned primarily with “trustworthiness”/integrity of data

◆ Multilateral security

– Chinese wall

- Developed for commercial applications



Bell-LaPadula model

- ◆ Simple security condition: S can read O if and only if $l_o \leq l_s$
- ◆ *-property: S can write O if and only if $l_s \leq l_o$
 - Why?
- ◆ “Read down; write up”
 - Information flows upward
- ◆ Why?
 - Trojan horse
 - Even with the right intentions, could be dangerous...



Basic security theorem

- ◆ If a system begins in a secure state, and always preserves the simple security condition and the *-property, then the system will always remain in a secure state
 - I.e., information never flows down...



Communicating down...

- ◆ How to communicate from a higher security level to a lower one?
 - Max. security level vs. current security level
 - Maximum security level must always dominate the current security level
 - Reduce security level to write down...
 - Security theorem no longer holds
 - Must rely on users to be security-conscious



Commercial vs. military systems

- ◆ The Bell-LaPadula model does not work well for commercial systems
 - Users given access to data as needed
 - Discretionary access control vs. mandatory access control
 - Would require large number of categories and classifications
 - Centralized handling of “security clearances”



Biba model

- ◆ Concerned with integrity
 - “Dual” of Bell-LaPadula model
- ◆ The higher the level, the more confidence
 - More confidence that a program will act correctly
 - More confidence that a subject will act appropriately
 - More confidence that data is trustworthy
- ◆ Integrity levels may be independent of security classifications
 - Confidentiality vs. trustworthiness
 - Information flow vs. information modification



Biba model

- ◆ Simple integrity condition: S can read O if and only if $I_s \leq I_o$
 - I_s, I_o denote the integrity levels
- ◆ (Integrity) *-property: S can write O if and only if $I_o \leq I_s$
 - Why?
 - The information obtained from a subject cannot be more trustworthy than the subject itself
- ◆ “Read up; write down”
 - Information flows downward



Security theorem

- ◆ An information transfer path is a sequence of objects $o_1, \dots, o_n$ and subjects $s_1, \dots, s_{n-1}$, such that, for all i , s_i can read o_i and write to o_{i+1}
 - Information can be transferred from o_1 to o_n via a sequence of read-write operations
- ◆ Theorem: If there is an information transfer path from o_1 to o_n , then $I(o_n) \leq I(o_1)$
 - Informally: information transfer does not increase the trustworthiness of the data
- ◆ Note: says nothing about secrecy...



“Low-water-mark” policy

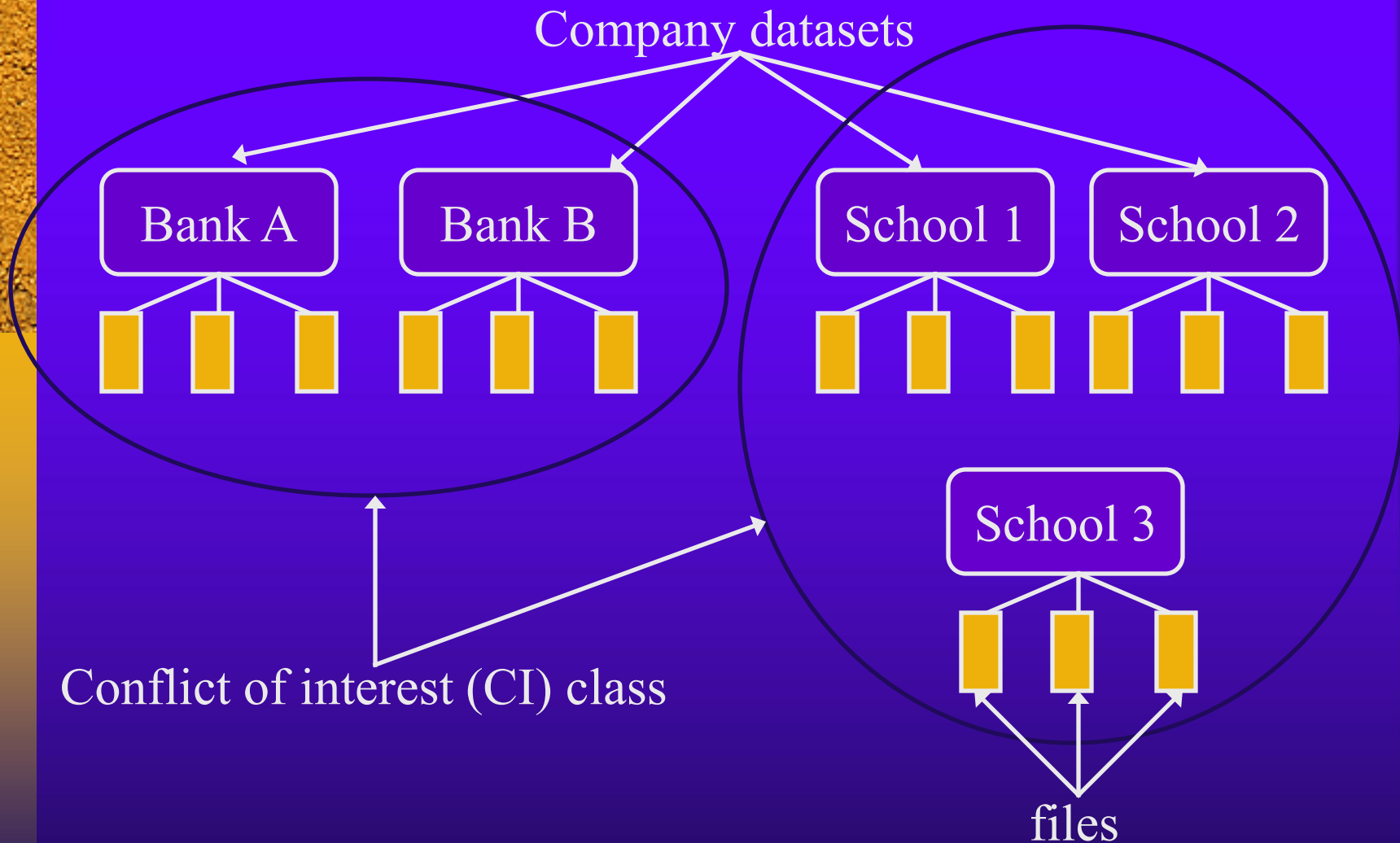
- ◆ Variation of “pure” Biba model
- ◆ If s reads o , then the integrity level of s is changed to $\min(l_o, l_s)$
 - The subject may be relying on data less trustworthy than itself
- ◆ If s writes to o , the integrity level of o is changed to $\min(l_o, l_s)$
 - The subject may have written untrustworthy data to o
- ◆ Drawback: the integrity level of subjects/objects is non-increasing!



Chinese wall

- ◆ Intended to prevent conflicts of interest
- ◆ Rights are dynamically updated based on actions of the subjects

Chinese wall -- basic setup

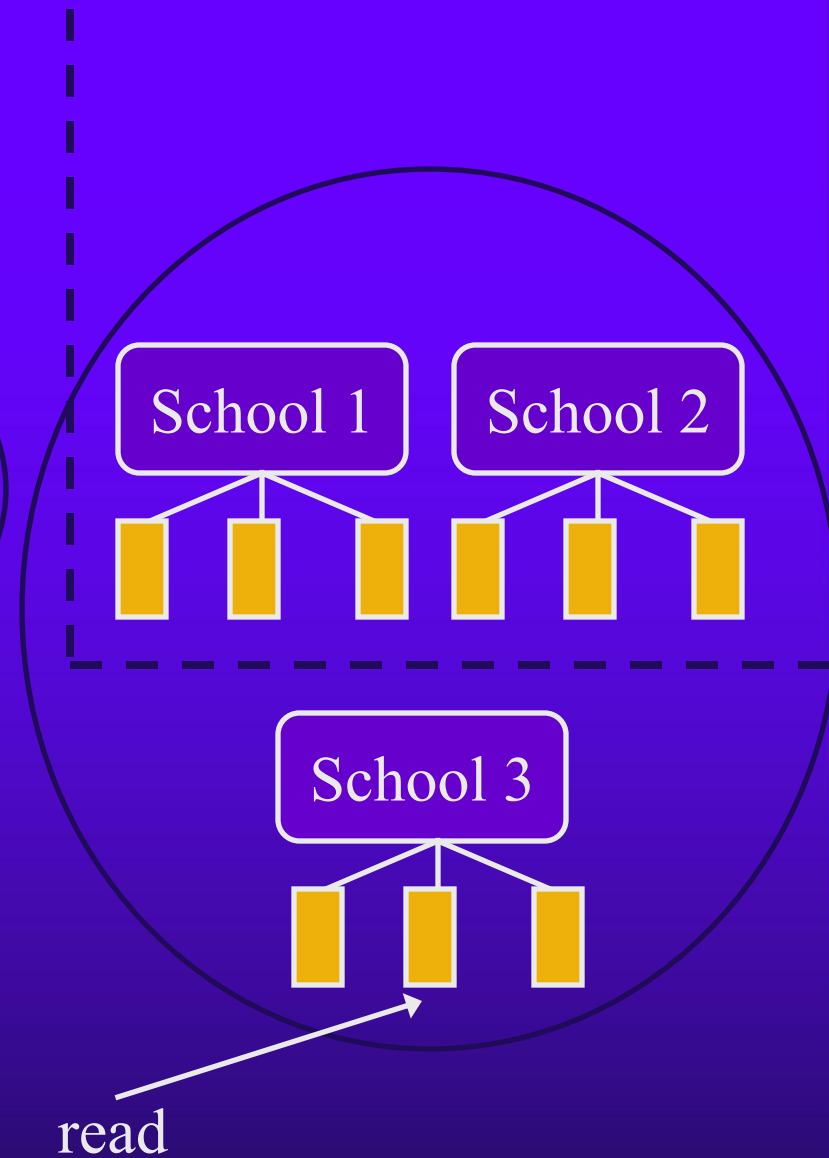
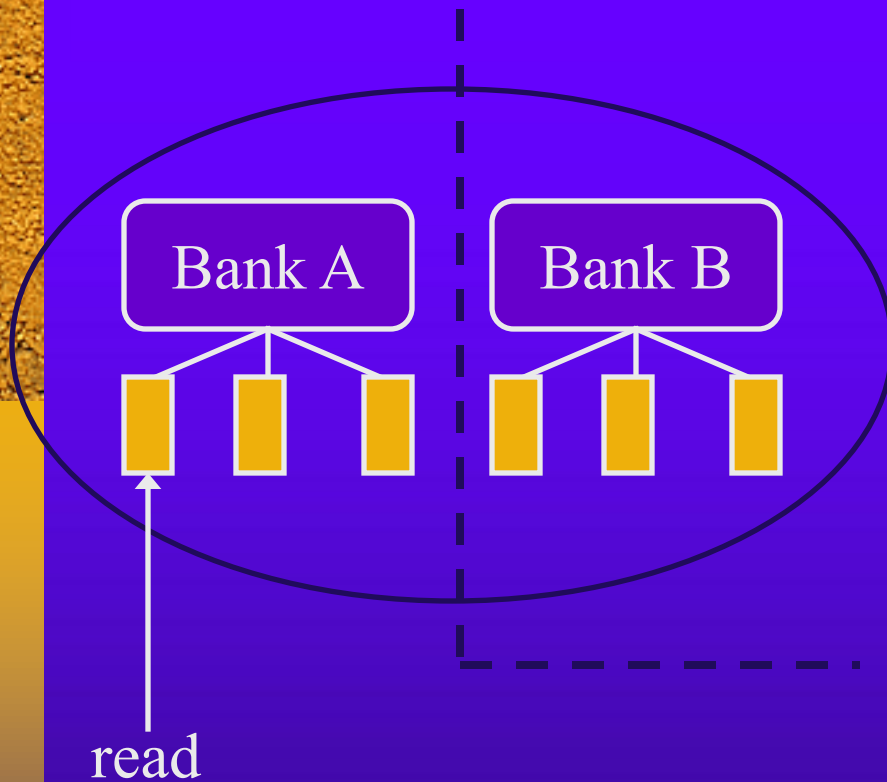




Chinese wall rules

- ◆ Subject S is allowed to read from *at most one* company dataset in any CI class
 - This rule is dynamically updated as accesses occur
 - See next slide...

Example





Chinese wall rules II

- ◆ S can write to O only if
 - S can read O and
 - All objects that S can read are in the same dataset as O
- ◆ This is intended to prevent an indirect flow of information that would cause a conflict of interest
 - E.g., S reads from Bank A and writes to School 1; S' can read from School 1 and Bank B
 - S' may find out information about Banks A and B!
- ◆ Note that S can write to at most one dataset...



Role-based access control



RBAC

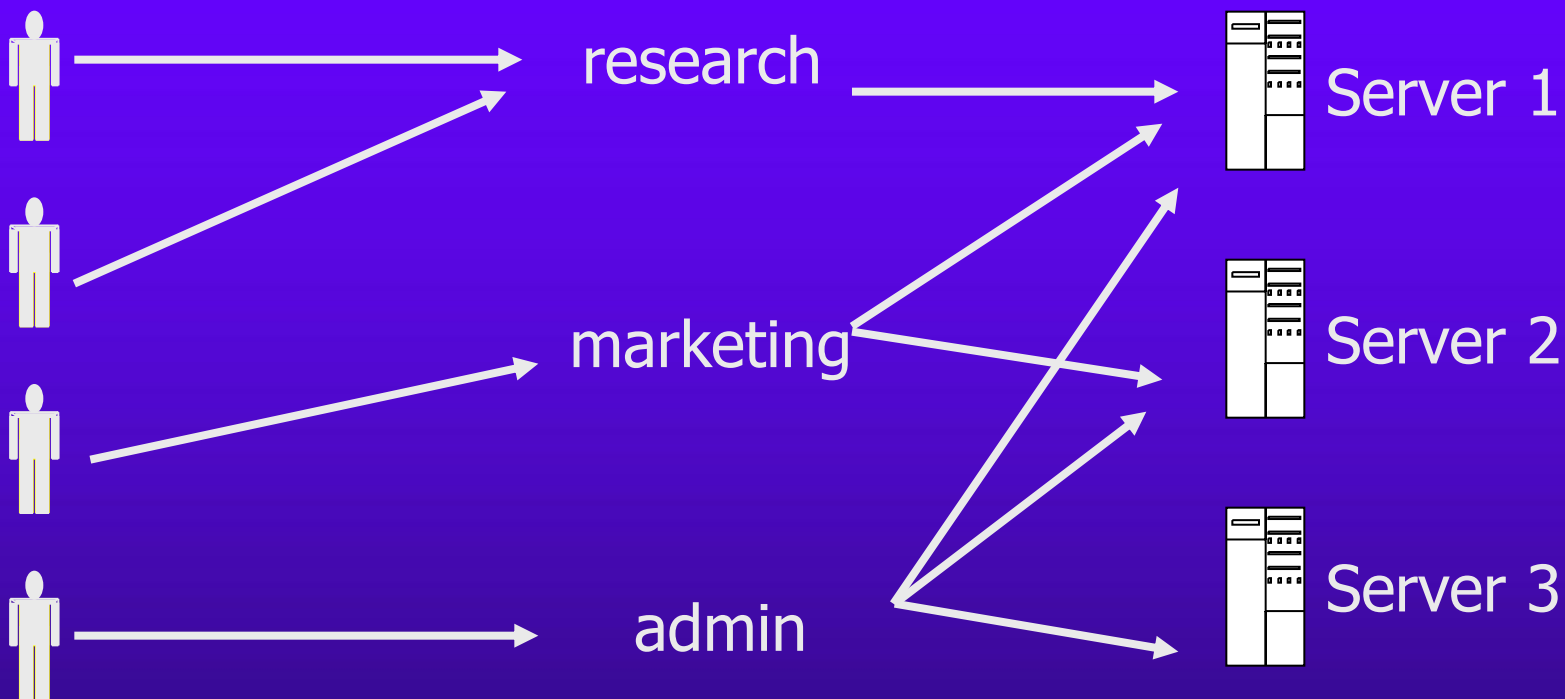
- ◆ Access controls assigned based on *roles*
 - Can use an access matrix, where “subjects” are roles
- ◆ Users assigned to different roles
 - Can be static or dynamic
 - A user can have multiple roles assigned
 - Can use “access matrix” with users as rows, and roles as columns
 - Will, in general, be more compact than a full-blown access control matrix
- ◆ Advantage: users change more frequently than roles

RBAC: basic idea

Users

Roles

Resources





Questions...

- ◆ Where might each of DAC, MAC, or RBAC make the most sense?



Code-based access control



Identity-based vs. code-based

- ◆ The access control policies we have discussed so far have all been *identity-based*
 - I.e., ultimately decisions come down to the identity of the principal/subject
- ◆ This works in ‘closed’ organizations
 - Principals correspond to known people
 - Organization has authority over its members
 - Users can be held accountable for their actions
- ◆ Does not work in ‘open’ settings
 - E.g., running code from the web



Code-based access control

- ◆ Determine rights of a process based on characteristics of the code itself, and/or its source
 - E.g., code downloaded from local site or remote site?
 - E.g., code signed by trusted source?
 - E.g., does code try to read from/write to disk?
 - E.g., does code contain buffer overflows?
 - Checked locally
 - ‘Proof-carrying code’



Difficulties

- ◆ Difficulties arise when one process calls another
 - E.g., remote process calls local process, or signed process calls an unsigned process
- ◆ Case 1: “trusted” g calls “untrusted” f
 - Default should be to disallow access
 - But g could explicitly delegate its right to f
- ◆ Case 2: “untrusted” f calls “trusted” g
 - Default should be to disallow access
 - But g could explicitly ‘assert’ its right
 - (cf. confused deputy problem)



Java 1 security model

- ◆ Unsigned applets limited to sandbox
 - E.g., no access to user's filesystem
- ◆ Local code unrestricted
 - Since Java 1.1, signed code also unrestricted
- ◆ Drawbacks
 - No finer-grained control
 - Code location not foolproof
 - Local filesystem on remote machine
 - Remote code that gets cached on the local machine



Java 2 security model

- ◆ *Byte code verifier, class loaders*
- ◆ *Security policy*
 - Grants access to code based on code properties determined by the above
- ◆ *Security manager/access controller*
 - Enforce the policy



Byte code verifier

- ◆ Analyzes Java class files (using, e.g., static type checking and data-flow analysis) to ensure certain properties are met
- ◆ E.g.,
 - No stack overflow
 - Methods called with arguments of appropriate type
 - No violation of access restrictions
- ◆ Note: these are static checks, not run-time checks



Class loaders

- ◆ Link-time checks performed when needed classes are loaded



Security policy

- ◆ Maps attributes of the code to permissions
 - Developers may define application-specific permissions
- ◆ May depend on the source code itself, as well as any code signers



Security manager

- ◆ The ‘reference monitor’ in Java
- ◆ Invoked at run-time to check the *execution context* (i.e., execution stack) against required permissions
 - Each method on the stack has a class; each class belongs to a *protection domain* indicating permissions granted to the class
- ◆ Security manager computes the intersection of permissions for all methods on the stack (‘stack walk’), and compares against required permissions
 - A method can also *assert* permissions, in which case prior callers are ignored



An example

```
f(){  
  foo;  
  g(); }
```

```
g(){ read, /tmp  
  bar;  
  doPrivileged...  
  h(); }
```

```
h(){  
  ... }
```

h	read, /tmp
g	read, /tmp
f	

$$\text{Perms} = \text{Perm}_h \cap \text{Perm}_g \cap \text{Perm}_f$$



Trusted Computing



Overview

- ◆ Secure hardware (*Trusted Platform Module*, or TPM) installed in computer
- ◆ Goals
 - Secure boot
 - Software verification
 - Attestation
 - Encrypted storage
- ◆ This is already deployed

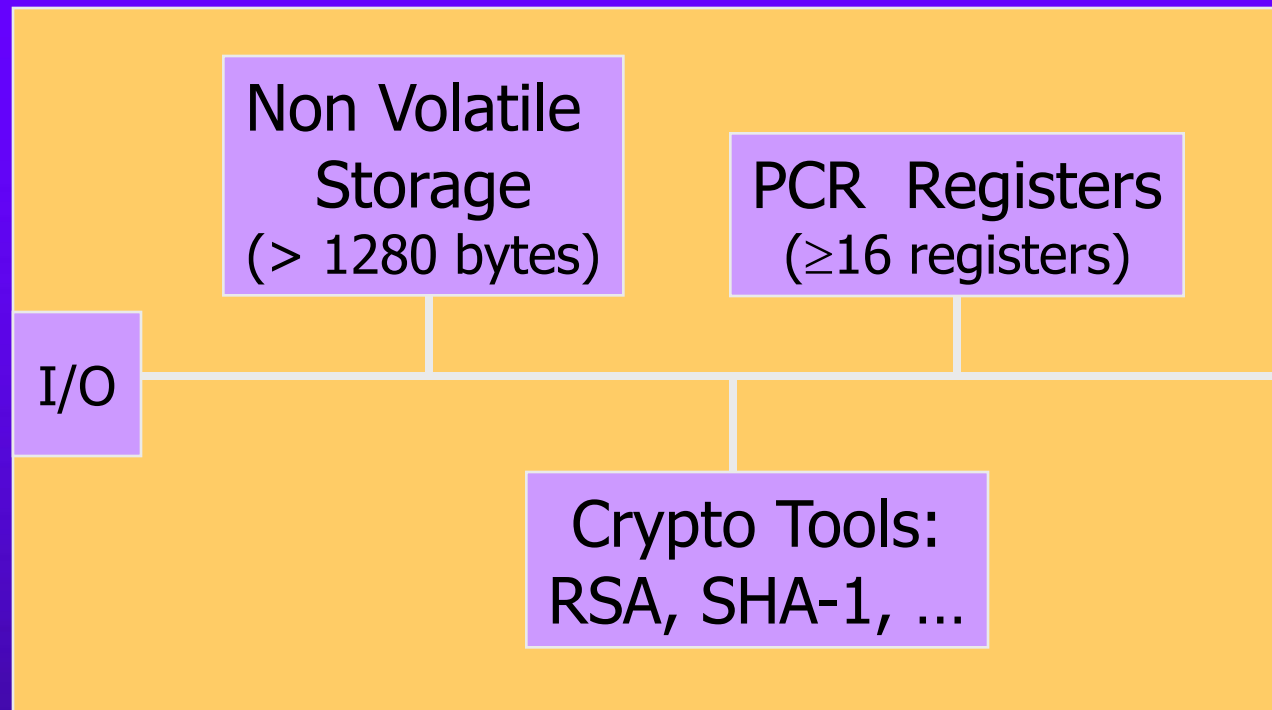


Disclaimer

- ◆ The intent of the following is to give the high-level ideas, rather than completely correct low-level details
- ◆ Full specification available on-line
 - TCG consortium



TPM chip





Non-volatile storage

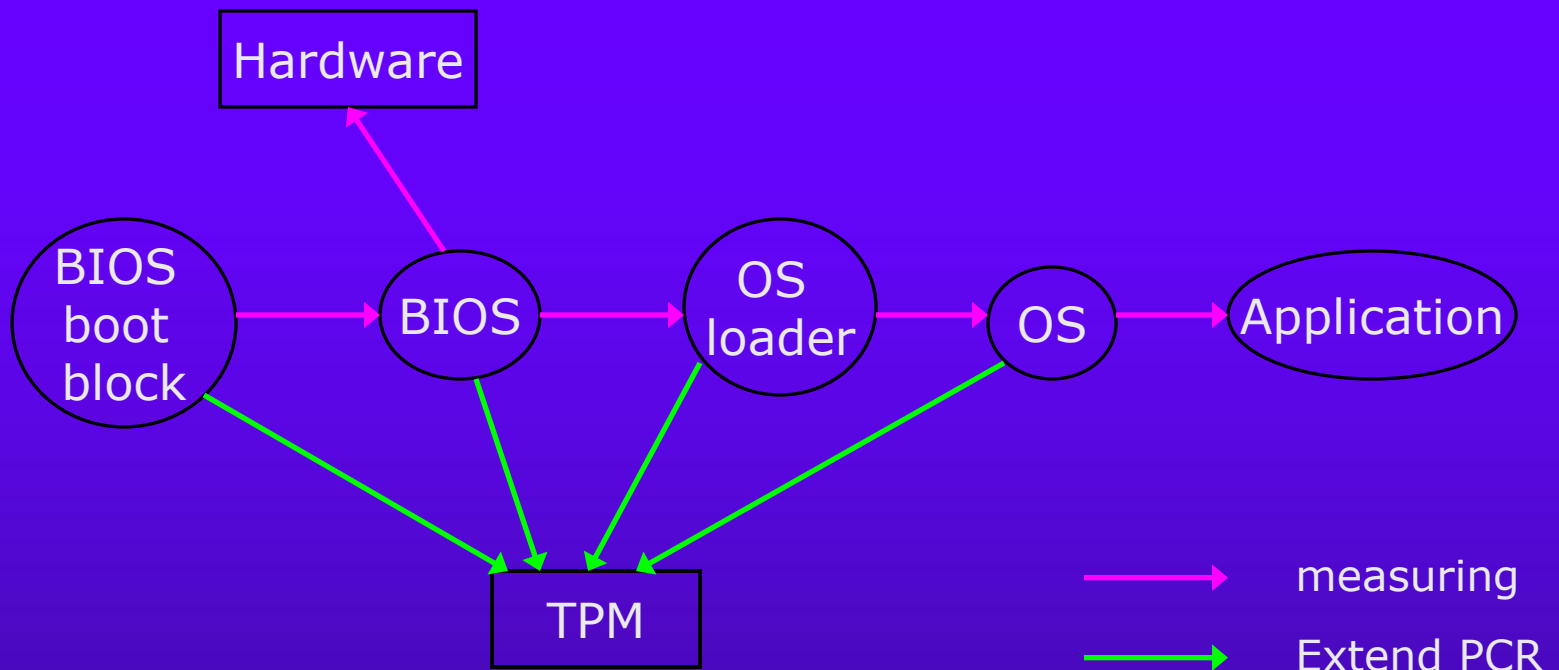
- ◆ Endorsement keys (EK) [RSA]
 - Created at manufacturing time, bound to computer
 - Signing keys; used for attestation
 - Cannot be changed (enforced by hardware)
 - Tamper-resistant; user cannot read or modify EK
- ◆ Storage root key (SRK) [RSA]
 - Created by user; can be changed
 - Used to encrypt data



PCR

- ◆ “Platform Configuration Registers”
- ◆ 20 bytes; hold SHA-1 output
- ◆ Can only be modified in two ways (enforced by the hardware):
 - TPM_Startup (initialize the contents of the PCR)
 - TPM_Extend(D): $PCR = \text{SHA-1}(PCR \parallel D)$
- ◆ Used to obtain an “image” of the loaded software...

PCM usage



- Collision resistance of SHA1 ensures “uniqueness”



What is this good for?

- ◆ Compare computed value with reference value
 - Secure boot
- ◆ Software validation
 - Verify signature before installing new software
 - All this verifies is the source
- ◆ Decrypt data
 - Decrypt only if in known (good) configuration
- ◆ Attestation
 - Prove to a third party that you are in a good configuration



Encrypted data

- ◆ Encrypt AES key K with SRK; encrypt bulk data with K
 - Hybrid encryption!
- ◆ When encrypting the AES key, embed current PCR value
 - E.g., $\text{Sign}_{EK}(\text{PCR}, \text{Enc}_{\text{SRK}}(K))$
 - (This is not actually the way it is done)
- ◆ When decrypting, check that the embedded value matches the current value
 - Refuse to decrypt if this is not the case!
- ◆ Can also incorporate a user password, etc.



Attestation

- ◆ Goal: prove to a remote party what software is running on my machine
- ◆ Applications:
 - Prove to company network that no viruses are running on my machine
 - Prove to another player that I am running an unmodified version of Quake
 - Prove to Apple that I am running iTunes...



Basic idea

- ◆ Sign PCR value with EK
 - Actually, sign with attestation identity key (AIK) validated with EK (ignore this for now)
- ◆ Assume third party knows EK
 - There is actually a PKI
- ◆ To prevent replay, use nonce provided by the third party
- ◆ Third party verifies signature; verifies that PCR corresponds to known “good” state



Controversy

◆ Loss of anonymity

- Signature using EK uniquely identifies the machine it came from
 - Some recent crypto proposals to address this
- Third parties can tell what software you are running

◆ Loss of control

- What if google says you need to have google desktop installed in order to use their search engine?
- What if Sony says you must use their music player to download their music?
- User can't access information on the machine they own